EXTERNAL WEB APPLICATION ASSESSMENT

Demo Retail — Security Assessment Report

Black-box and authenticated testing of the customer web app and REST API.

CLIENT	SCOPE	METHOD	SCORING
Demo Retail (fictional)	**app.example.com** **api.example.com**	**OWASP WSTG v4.2**	**CVSS v3.1**

Executive summary

We identified **five** vulnerabilities. One critical issue allowed an unauthenticated attacker to read the full product and customer database through the search feature. It was fixed within 48 hours of reporting and verified in retest. Four of five findings are now closed; one is awaiting retest.

1	1	2	1
CRITICAL	HIGH	MEDIUM	LOW

Findings overview

ID	FINDING	SEVERITY	CVSS	STATUS
ZB-01	SQL injection in product search parameter A03:2021 Injection	CRITICAL	9.8	Fixed
ZB-02	Broken object-level authorisation on /api/orders/{id} A01:2021 Broken Access Control	HIGH	8.1	Fixed
ZB-03	No rate limiting or lockout on the login endpoint A07:2021 Identification & Auth Failures	MEDIUM	6.5	Retest
ZB-04	Session cookie missing Secure and SameSite flags A05:2021 Security Misconfiguration	MEDIUM	5.4	Fixed
ZB-05	Server software version disclosed in response headers A05:2021 Security Misconfiguration	LOW	3.7	Fixed

Overall risk after remediation

Low. No critical or high findings remain open. We recommend adding automated dependency and SAST scanning to the CI pipeline to prevent regressions.

FINDING ZB-01

SQL injection in product search

CRITICAL · 9.8

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

AFFECTED	GET https://app.example.com/search?q=
CATEGORY	OWASP A03:2021 Injection · CWE-89
STATUS	**Fixed · verified in retest**

Description

The `q` parameter was concatenated directly into a SQL query. Adding a quote character changed the query logic, so the application returned every row in the products table instead of matching items.

Evidence (sanitised)

```
GET /search?q=shoes%27+OR+%271%27%3D%271 HTTP/1.1
Host: app.example.com

HTTP/1.1 200 OK
X-Result-Count: 4812      # expected: 37
```

Impact

An unauthenticated attacker could read, and potentially modify, data in the shared database, including customer names, emails and order history.

Remediation

- Use parameterised queries or the ORM query builder for every database call.
- Validate search input against an allow-list of characters and length.
- Run the database user with read-only rights for search queries.
- Add SAST (e.g. Semgrep) to CI to catch string-built SQL before release.

Retest

The same request now returns the expected 37 results and the payload is treated as plain text. The query uses bound parameters. Closed.

References

- OWASP Top 10 2021 – A03 Injection
- OWASP Cheat Sheet – SQL Injection Prevention
- CWE-89: Improper Neutralization of Special Elements in SQL